Bento.rs - Project Report

github.com/HemanthKR07/bento.rs

A rootless, daemonless, low level container runtime for Linux written in Rust that aims to be compliant with the OCI Runtime Spec.

Hemanth KR, 4th year undergraduate student at PES University, Bangalore INDIA.
github.com/HemanthKR07

This report has been drafted completely from my notes prepared while learning. And AI was only used to check the correctness of the report and grammar.

Link: *app.notion.com/p/Bento-rs-19c411965540800888a2ec168b59a9b8*

---

1. Introduction

bento.rs is a rootless, daemonless, low-level container runtime for Linux, written in Rust, that aims to be compliant with the Open Container Initiative (OCI) Runtime Specification. The project began as a collaborative open-source effort under homebrew-ec-foss/bento.rs. This report documents the concepts behind container runtimes and describes, in detail, the three areas I contributed to: rootless filesystem isolation, the OverlayFS union filesystem, and seccomp syscall filtering.

The goal of bento.rs is to offer a minimal, fast, and secure alternative to existing container runtimes such as runc and crun, while exploring how far a memory-safe language like Rust can be pushed for low-level, kernel-facing systems programming.

---

2. Background and Core Concepts

Before describing the implementation, it is useful to define the building blocks of container technology that bento.rs relies on.

2.1 Containers and Container Runtimes

Container: A lightweight, standalone software package that bundles an application together with everything it needs to run - code, runtime, system libraries, and settings. Containers are isolated from one another, so applications do not conflict, and each can be managed independently.

Container runtime: The software responsible for the actual life cycle of a container- pulling the image from a container registry, unpacking it, creating the isolated environment, running the process, monitoring its health, and cleaning up resources once the container exits.

A container's execution generally follows this sequence:

- Container creation
- Initialization of the environment based on the container image
- Running the container - starting the application, ensuring it functions correctly, and monitoring/restarting it if it fails
- Cleanup of resources once the container is no longer in use

2.2 Why Containerization Matters

- Isolation - dependencies of one application never conflict with another.
- Portability - the application and its dependencies travel together, giving consistent behaviour across machines.
- Scalability - containers can be scaled up or down quickly, typically managed by orchestrators such as Kubernetes, Docker Swarm, or Apache Mesos.
- Resource efficiency - containers share the host kernel, so overhead is limited to the application's own libraries and dependencies, unlike a full guest OS.
- Microservices - since each container runs independently, applications can be split into small, independently deployable services that communicate over APIs.

2.3 Containers vs. Virtual Machines

A Virtual Machine (VM) virtualizes hardware and needs a complete guest operating system, including its own kernel, for every instance. A Hypervisor is the software layer, running on the host, that lets several such guest OS instances share the same physical hardware. Virtualization therefore allows multiple operating systems to run simultaneously on one machine, each with its own kernel.

Containers, in contrast, perform OS-level virtualization: every container shares the host's kernel and only isolates the process's view of the system. This is what makes containers dramatically lighter and faster to start than VMs.

2.4 Linux Namespaces

A namespace is a Linux kernel feature that gives a process (or a group of processes) its own isolated view of a particular kind of global system resource. From inside a namespace, a process appears to have the whole machine to itself, even though it is really just one of many processes on a shared host. The main namespace types are:

- User - lets a container have its own set of UIDs/GIDs, separate from the host's. A process can be root inside the container while being an unprivileged, non-root user on the host. Without a user namespace, a root process inside a container is also root on the host, which is a serious security risk.
- PID - isolates process IDs, so a container can only see its own processes.
- Network - gives each container its own network interfaces, routing table, IP addresses, socket listings and firewall rules.

- Mount - gives each container its own filesystem view. When a mount namespace is created (CLONE_NEWNS), the child's mount table starts as a copy of the parent's; any mount or unmount the child performs afterwards affects only its own namespace, not the parent's
- IPC - isolates inter-process communication (message queues, shared memory, semaphores) so a container's processes cannot signal or share memory with processes outside it
- UTS (UNIX Time-sharing System) - gives a container its own hostname and NIS domain name, independent of the host's.
- Cgroups (as a namespace/controller) - limits how much CPU, memory and other resources a container's processes can consume; a process inside a container cannot see or change these limits from within.

Useful commands and syscalls while working with namespaces:

- unshare - detaches the calling process from namespaces it currently shares with its parent, based on a bitmask of CLONE_NEW* flags.
- clone() - creates a new child process and, depending on the flags passed, places it directly into new namespaces at creation time.
- setns() (used by the nsenter command) - moves an already-running process into an existing namespace.
- /proc/<pid>/ns - inspecting this directory shows which namespaces a given process belongs to.

2.5 The OCI Standard

The Open Container Initiative (OCI) defines three specifications that most modern runtimes implement:

- Runtime Spec - defines the configuration, execution environment, and lifecycle of a container.
- Image Spec - defines the format of a container image.
- Distribution Spec - defines how images are pushed to and pulled from a registry.

When an OCI image is unpacked, the result is a filesystem bundle containing a config.json file (which describes everything the runtime needs to run the container - environment variables, the command to execute, namespaces, cgroups, lifecycle hooks, filesystem mounts, and user/group IDs) and, optionally, a rootfs directory that becomes the container's root filesystem.

Rootless means the runtime - not just the containers it launches - can be created, run and managed entirely by an unprivileged, non-root user. Daemonless means there is no long-running background process (such as dockerd) managing containers; if a daemon process is compromised, an attacker potentially gains access to every container it manages, plus the host, because that daemon typically runs as root. Removing the daemon removes that single point of failure.

2.6 Other Related Concepts

seccomp (Secure Computing Mode): a Linux kernel feature that acts like a firewall for system calls, restricting the set of syscalls a process is allowed to make. It is one of the primary contributions covered in this report and is discussed in depth in Section 4.

LXC and VM-based containers: LXC is a runtime aiming for an experience close to a VM but with far less overhead, using the full kernel and hardware. Some 'container' runtimes (e.g. Kata Containers) instead launch a lightweight VM per container for stronger, hardware-enforced isolation: the OCI bundle is mounted inside a purpose-built micro-VM and commands run in the guest kernel via an in-VM agent, rather than in a namespace on the host kernel. In a conventional OCI runtime such as bento.rs, by contrast, the container's command executes directly on the host kernel, inside an isolated namespace — there is no guest kernel involved.

---

3. Why Rust

bento.rs is implemented in Rust rather than C (the language runc and crun are written in), for several reasons:

- Rust catches classes of bugs - null pointer dereferences, buffer overflows, use-after-free - at compile time rather than at runtime.
- Rust's ownership and borrowing model catches data races at compile time.
- Rust is strongly typed, which keeps the codebase cleaner and easier to reason about.
- Rust provides memory safety without a garbage collector, avoiding the class of security issues C runtimes are historically prone to.

Rust is not yet the dominant language in the container ecosystem, but its safety guarantees make it well suited to a security-sensitive project like an OCI runtime.

---

4. Project Overview

4.1 Repository Structure

```
crates/
├── bento-cli/
│   ├── src/
│   │   └── main.rs             # CLI entry point (built using clap)
└── libbento/
    └── src/
        ├── process.rs          # Container lifecycle and fork/clone orchestration
        ├── fs.rs               # Root filesystem setup, pivot_root, and mount management
        ├── overlayfs.rs        # OverlayFS layer creation and management
        ├── seccomp.rs          # libseccomp filter creation and loading
        └── syscalls.rs         # Low-level Linux syscall wrappers
```

4.2 Feature Status

Component	Status
Namespace isolation (PID, mount, UTS, user, IPC)	Done
Rootless rootfs setup via pivot_root	Done
OverlayFS union filesystem (lower/upper/work/merged)	Done
Seccomp syscall filtering (libseccomp, BPF)	Done
create / start / list	Done
state / kill / delete / spec	Stubbed
PTY support	In progress
Cgroups	Planned

4.3 Usage

```
cargo build –release                                  # Build the project in release mode

mkdir -p test-bundle/rootfs/bin                       # Create a minimal OCI bundle
cp /bin/sh test-bundle/rootfs/bin/

# (Place config.json inside the test-bundle)          # Add the OCI configuration file

cargo run -- create --bundle test-bundle my-container  # Create a container from the bundle

cargo run -- create --bundle test-bundle my-container --overlayfs
# Create a container with OverlayFS enabled

cargo run -- start my-container                        # Start the created container

cargo run -- list                                      # List all existing containers
```

---

5. My Contributions

My work on bento.rs covered three areas: rootless filesystem isolation (rewritten and completed from an initial scaffold), OverlayFS support (designed and implemented independently, not present in the upstream project), and seccomp syscall filtering (completed and maintained from an in-progress upstream pull request). Each is described below, including the underlying kernel concepts and the reasoning behind implementation decisions.

5.1 Rootless Filesystem Setup

Rootless container creation means an unprivileged user can create, run and manage containers without needing root on the host. This relies on Linux user namespaces to remap UIDs/GIDs, and on careful use of mount namespaces and pivot_root to switch the process into the container's own root filesystem without ever needing host-level root privileges.

5.1.1 Getting PID 1 inside the new namespace

A key implementation detail is how the first process inside the container is created. It is tempting to think the runtime could just set up namespaces on the same process it is already running in, but Linux namespace semantics do not allow that: to actually isolate a process's view of PIDs, mounts, and the hostname, a new process has to be created inside the new namespaces from the start.

unshare() moves the calling process's namespace context, but the calling process itself stays where it is - it does not become PID 1 in the new PID namespace. Instead, the first process forked after the unshare() call is the one placed inside the new PID namespace, and that child becomes PID 1 there. Since PID 1 has special behaviour that a container's init process depends on (it does not get killed by ordinary signals the way other processes do, it inherits orphaned child processes, and its death naturally ends the container), it matters that this first forked child really is PID 1.

Solution: bento.rs first calls unshare() with CLONE_NEWUSER alone to create the user namespace, then performs UID/GID mapping while still in the parent's other namespaces. Once mapping is complete, it calls unshare() again with CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS combined to create the remaining namespaces. Because unshare() does not move the calling process into the new PID namespace, bento.rs then calls fork(): that forked child is the first process created after the PID namespace exists, so it is guaranteed to become PID 1 inside it. This ordering - user namespace and ID mapping first, then unshare() for the remaining namespaces, then a fork() to spawn the init process - is the core of how bento.rs achieves rootless container creation.

5.1.2 Switching the root filesystem (pivot_root)

Once the container process exists inside its own mount namespace, it still needs to be confined to the container's own root filesystem so it cannot see or modify the host's files. This is done with pivot_root, which swaps the process's root directory for a new one and moves the old root out of the way so it can be unmounted.

Before calling pivot_root, mount propagation for the new mount namespace is reset to private and recursive (MS_REC | MS_PRIVATE), so that any mount or unmount events inside the container do not propagate back out to the host, and vice versa.

The general pattern is:

```
mkdir old_root pivot_root(".","old_root")      #inside the container's rootfs directory

cd / umount("old_root", MNT_DETACH)          # current dir becomes '/', old '/' moves to old_root

rmdir("old_root")                              # host filesystem is no longer reachable
```

After this sequence, the process's '/' really is the container's rootfs - its /bin, /etc and /lib - and the host filesystem it used to see at the old path is unmounted and unreachable. This is what actually isolates the container from the host's files, on top of the namespace isolation from Section 5.1.1.

This entire flow - user namespace setup, UID/GID mapping, the combined clone() call, and pivot_root - lives in libbento's process.rs and fs.rs modules.

5.2 OverlayFS Union Filesystem

OverlayFS was designed and implemented independently for bento.rs - it was not present in the upstream project. It gives each container a writable root filesystem while keeping the base container image completely untouched and reusable across containers.

OverlayFS works by combining several directories into a single unified view, using four layers:

- Lower - a read-only, immutable directory. This is the container's base filesystem (its /bin, /etc, and so on), taken from the container image and referenced via the bundle's config.json. It is the starting point for everything the container sees.
- Upper - a writable directory where every modification made inside the running container actually lands: new files, edited files, and 'whiteout' markers that record file deletions from the lower layer. The lower layer itself is never modified.
- Work - an internal directory used purely by the kernel for bookkeeping during write operations. It starts out empty, lives on the same filesystem as the upper directory, is never visible to the container, and is a hard technical requirement of OverlayFS - without it, the mount fails.
- Merged - the actual OverlayFS mount point: a single, unified view that combines lower and upper. Files from lower are visible unless they were changed or deleted in upper, in which case upper's version (or the deletion) takes precedence. This merged directory is what becomes the container's root filesystem, mounted in with pivot_root.

A typical on-disk layout used by bento.rs looks like this:

```
/path/to/container/rootfs/   # lower (read-only image contents) /var/lib/bento/container56/

upper/                       # writable layer   work/           # kernel bookkeeping

merged/                      # mount point -> becomes the container root     proc/     sys/     dev/
```

In short: lower holds the immutable base files, upper holds only new or changed files, work is reserved for the kernel, and merged is what the container actually sees and treats as a single, ordinary, mutable filesystem. In bento.rs this is implemented in libbento's overlayfs.rs and can be enabled at container-creation time with the --overlayfs flag; when it is not passed, the runtime falls back to using the bundle's rootfs directly.

5.3 Seccomp Syscall Filtering

Seccomp (Secure Computing Mode) is a Linux kernel feature that lets a process restrict itself to a small, explicit set of allowed system calls. It acts as a firewall between the container's process and the kernel, blocking any syscall that could be used to escape isolation or attack the host, even if the process running inside the container is compromised. This work was originally started as an upstream pull request and was completed and is maintained in bento.rs.

5.3.1 Seccomp Modes

- Strict mode - the process may only call read(), write(), _exit() and sigreturn(). This is extremely restrictive and rarely usable in practice, set via prctl(PR_SET_SECCOMP, SECCOMP_MODE_STRICT).
- Filter mode - the mode bento.rs uses. A BPF (Berkeley Packet Filter) program defines, per architecture, which syscalls are allowed, denied, logged, or trapped, set via prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, prog), where prog points to a compiled sock_fprog filter.

Before installing a filter, the process must call prctl(PR_SET_NO_NEW_PRIVS, 1). This stops the process (and anything it later execs) from gaining new privileges, which would otherwise require the CAP_SYS_ADMIN capability - something a rootless, unprivileged runtime does not have.

At the kernel level, each syscall a filtered process makes is packaged into a seccomp_data structure (containing the syscall number, the instruction pointer, and the syscall's arguments) and handed to the loaded BPF program, which returns one of: allow, deny with an error, kill the process, log the call, or trap (deliver SIGSYS, a 'bad system call' signal, to the process for custom handling).

5.3.2 How bento.rs Builds the Filter

bento.rs uses libseccomp, a Rust crate providing safe bindings to the C libseccomp library, to build and load BPF filter programs, avoiding the need to hand-write BPF bytecode. The flow is:

- Parse the bundle's config.json and read the OCI linux.seccomp section: the defaultAction, the target architectures, and the per-syscall rules.
- Deserialize that JSON into Rust structs (SeccompConfig, SyscallRule), and use libseccomp's ScmpFilterContext, ScmpAction, ScmpSyscall, and ScmpArch types to build the filter programmatically, matching what the OCI spec describes.
- Validate the parsed configuration for structural completeness before it is ever loaded: at least one target architecture must be present (this is a hard failure if missing), while an empty syscall-rule list or the absence of an allowed exit/exit_group syscall only produce warnings rather than aborting. Unknown or unrecognized syscall names are skipped individually, with a warning, at rule-construction time rather than at validation time.

- Load the compiled filter into the kernel via ScmpFilterContext::load(), which must happen before the container's init process actually execs the application - this matches how runc and crun apply seccomp policy defined directly in the OCI spec (as opposed to higher-level tools like Podman, which build their own JSON on top and translate it down via libseccomp before it ever reaches the runtime).

Tools such as strace are useful for debugging exactly which syscalls a process makes, which is valuable both when writing a seccomp policy and when diagnosing why a filtered process failed unexpectedly.

Note: the filter-construction and validation logic above is implemented on main. The integration point that actually invokes SeccompFilter::apply() inside the container init process before exec currently lives on the dev-seccomps branch and is in progress.

---

6. Roadmap

- Implement state, kill, delete and spec commands (currently stubbed).
- Add cgroups support for CPU and memory limits.
- Finish PTY-based interactive containers.
- Work towards passing the OCI conformance test suite.

---

7. Conclusion

Working on bento.rs's rootless filesystem setup, OverlayFS, and seccomp filtering meant going well below the level most container tooling is used at — into namespace semantics, pivot_root, union filesystem layering, and BPF-based syscall filtering. Building these in Rust, on top of the OCI Runtime Spec, gave a very concrete, hands-on understanding of how container runtimes actually isolate a process from its host, and why that isolation has to be built up carefully, one kernel primitive at a time, rather than assumed.

---

8. Attribution

- Rootless filesystem isolation — initially scaffolded together with a collaborator (Alex-Hunterz), then rewritten and completed by the author.
- Seccomp — implementation started upstream as a pending pull request, then completed and is maintained by the author.
- OverlayFS — designed and implemented independently by the author; not present in the upstream project.

Repository: github.com/HemanthKR07/bento.rs (forked from homebrew-ec-foss/bento.rs). Licensed under the MIT License.
